

Assignment 1: Initial Thoughts and System Design (A1)

QueueSmart – Smart Queue Management Application

1. Initial Thoughts

QueueSmart will be designed as a web application that aims to help organizations manage waiting queues more efficiently, while giving users clearer visibility about their position and expected wait-time.

a. Who are the main users of the system?

There will be two categories of users in QueueSmart, Users, and Administrators. Possible users include students, customers, and patients who want to join a virtual queue or schedule a service without having to wait in a physical line. Administrators will work with QueueSmart and Users/Customers to provide them with their service as seamlessly and efficiently as possible.

b. How will users and administrators interact with the application?

Both users and administrators will be able to register and login to the application. For users, logging in will allow them to browse and select a service to join a queue. They should be able to see their estimated wait-time, current position in the queue, leave the queue, and receive notifications when their turn is approaching, as well as once their wait is over. Admins should be able to create, manage, and update services, give services a priority level, monitor active queues, serve or remove queue entries, and view basic usage statistics and queue history, like queue length, number of users served, and average wait-time.

c. What are the most important features?

The most important features for QueueSmart are reliable queue management, clear wait-time information for their service, role-based access, service management, and notifications. The ordering of the queue will consider both the arrival time and priority of the service, where generally users who arrived earlier should be served earlier and higher priority services may move ahead if appropriate. The backend will use the expected service time and number of people ahead of the user to compute the estimated wait-time for the user.

d. What challenges do you anticipate?

Some challenges that we expect are being able to accurately determine wait-time, as actual service times could vary. The service may take longer depending on the time of day, the workers present, etc. Additionally, queue information can change rapidly in cases where a user leaves the queue, admins change priorities, or a service takes longer than expected.

Notifications must be sent at the right times without sending any sort of duplicate or unnecessary information. The system must also keep a fair queue while still supporting the higher priority levels. QueueSmart should also be designed with the issue of scalability in mind so that multiple active queues and users can be handled simultaneously without significantly reducing performance or reliability of the application. The program crashing/being dysfunctional can cause major issues for companies using QueueSmart. Finally, authentication and authorization must ensure that normal users cannot access features that are administrator only.

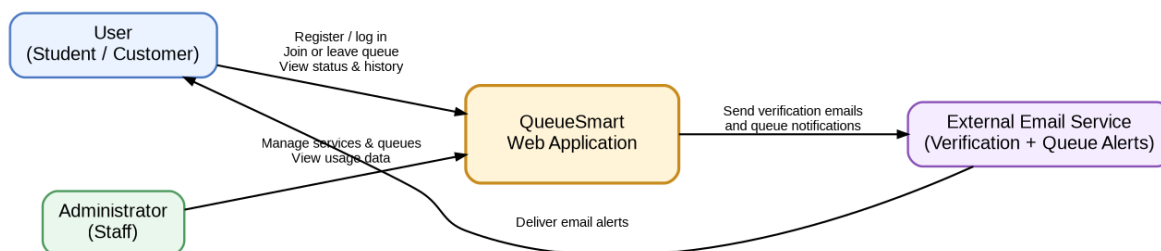
2. Development Methodology

For this project we will be using the Scrum development methodology. Scrum is an agile framework that divides the work into multiple sections called sprints. This allows us to build the project through iterative updates while giving us the flexibility of adjusting our design at any point during the process if changes are required. This is ideal for our project which is split into multiple assignments that require us to build our application piece by piece. Every work cycle will require us to evenly split the work amongst our group, execute our plan, receive feedback, and continuously improve our application.

By using a shared repository on GitHub we can coordinate our work efficiently and have a record of our individual contributions for each cycle of the project. With everyone involved in every cycle, everyone becomes familiar with every section of the application which facilitates communication within our group.

3. High-Level Design / Architecture

3.1 System Context Diagram



System Context Explanation

The User represents a student or customer who interacts with QueueSmart to register or log in, join or leave a queue, and view their current queue status and history. Users receive updated queue information through the application and may also receive email alerts when their queue status changes or their turn is approaching.

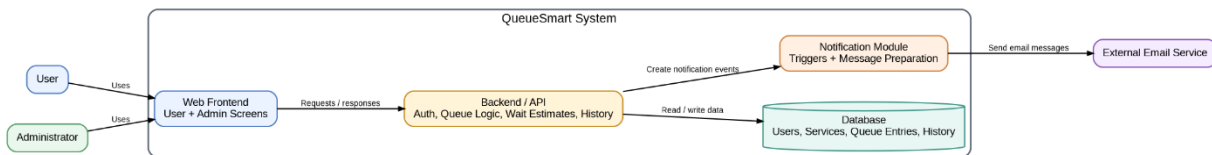
The Administrator represents staff members who use QueueSmart to manage services and active queues. Administrators can create and update services, monitor queue activity, manage queue entries, and view basic usage data and queue history. Administrator access is

separate from regular user access so that management features are only available to authorized staff.

The QueueSmart Web Application represents the complete QueueSmart system at the context level. It handles interactions from both users and administrators, including authentication, queue participation, service management, queue status, and history. In the System Context Diagram, QueueSmart is shown as a single system rather than its individual internal components.

The External Email Service is responsible for delivering emails generated by QueueSmart. The application will send verification emails and queue notification requests to the external service, which then delivers alerts to users. The email service is handled by an external provider rather than directly by the web application. All queue rules, user roles, service management, history, and wait-time calculations remain inside QueueSmart.

3.2 Optional Container Diagram



Container Diagram Explanation

The Web Frontend provides interfaces to both the regular users and administrators use. It displays queue position, estimated wait-time, history, available services, and administrator management tools. The frontend sends user actions directly to the backend, without directly changing application data.

The Backend/API contains all main application logic. It authenticates users, manages services, processes join and leave requests, checks whether a user has the permissions of a regular user or an administrator, orders queue entries by arrival time and priority, computes estimated wait-times for users requests, prepares basic usage statistics, and records the queue history.

The Notification Module receives notification events directly from the backend. For instance, when a user becomes close to the front of the queue, the backend may trigger a notification event. This notification module can then use the external email service to deliver the message to the user. Additionally, in-app notifications can be shown directly through the web application.

The Database stores persistent information, such as user accounts, services, roles, active queue entries, queue participation history, and priority information. Keeping this information in one data store allows the system to maintain queue state, even if a user closes their browser.

4. Major Component Interaction Summary

A typical QueueSmart interaction works like this:

1. User or Administrator logs into the web application of QueueSmart.
2. Web frontend sends request to the backend.
3. Backend verifies the user's account and role.
4. When user joins a queue, backend creates a queue entry for that user and stores updates queue information it in the database.
5. Backend determines user's position using arrival time and priority, and estimates wait-time using expected duration of selected service.
6. Updated queue information returned to frontend and displayed to user.
7. When user's turn is coming up, backend triggers notification module, sending an email notification through the external email service.
8. When service completed or user leaves queue, system updates active queue and saves the result in history.
9. Administrators can view current queue state and basic usage statistics using information stored in the database.

This design keeps the web application simple enough for the first stage of the project, and leaves room for later assignments to define things like the UI, APIs, and database structure with more detail.

5. Team Contributions

Group Member	Contribution	Discussion Notes
Tyler Landgrebe	Designed the optional Container Diagram, wrote the component interaction explanation, and integrated/proofread the final PDF submission. Touched up and cleaned up document.	Focused on internal component responsibilities and consistency of the final document.
Chase Beeler	Designed the System Context Diagram and wrote the System Context explanation, including the system boundary and external email service.	Focused on high-level architecture and external interactions.

Jeremy Yang	Wrote the Initial Thoughts section, defined the main users and their interactions, identified key features, and reviewed the user-side flows in the System Context Diagram.	Focused on requirements from the regular-user perspective and queue usability.
Alan Garcia	Wrote the Development Methodology section, defined the Agile/Scrum workflow, described the GitHub collaboration process, and documented major design challenges.	Focused on how the team will organize work across A1–A4 and the final project.